

Enterprise E-Commerce Predictive Analytics Project

Technical Documentation & Executive Overview of the Code, Data Pipeline, and Machine Learning Model

Project Handover Documentation

Enterprise E-Commerce Predictive Analytics

1. Project Idea & Objective

This project delivers a complete analytical pipeline for sales data belonging to a large-scale e-commerce operation that sells across multiple online marketplaces (e.g., Amazon and similar channels). Raw transactional data is pulled from its source layer, cleaned and merged into a single unified dataset, enriched with a wide range of engineered business and time-based features, and finally used both to compute executive-level KPIs and to train a machine learning model that automatically detects abnormal patterns (anomalies) in revenue, profit, orders, and related performance metrics — without relying on manual, row-by-row inspection of reports.

The core motivation behind this system is operational efficiency: e-commerce businesses generate thousands of daily data points across products, dates, and marketplaces, and it is practically impossible for an analyst to manually scan every row for suspicious spikes, drops, or irregularities. By training an unsupervised anomaly detection model directly on the enriched feature set, the system flags the small fraction of records that truly deserve human attention, ranks them by severity, and surfaces them through clear visual dashboards.

Business value: The pipeline gives the management and analytics team an early-warning system capable of catching sudden, unexpected shifts in sales, profitability, or conversion behavior — whether a surprising spike or a sharp decline — early enough to investigate the root cause and react before it affects quarterly results.

2. Data Sources & Structure

The project follows a **Medallion Architecture** (Bronze / Silver / Gold layers), reading directly from the clean, analytics-ready **Gold Layer**. Four core tables are loaded directly from CSV files hosted in the project's data repository and then combined into a single working dataframe:

Table	Description
fact_sales	The central fact table: daily revenue, orders, units, sessions, clicks, impressions, PPC cost/orders/sales, FBA fees, promotions, refunds, net profit, net margin, and net ROI at the product/marketplace/date grain
dim_product	Product dimension table containing descriptive attributes for every SKU
dim_marketplace	Marketplace dimension table describing each sales channel
dim_date	Calendar/date dimension table used to enrich each transaction with time context

These four tables are merged sequentially — `fact_sales` with `dim_product` on `Product_Key`, then with `dim_marketplace` on `Marketplace_Key`, and finally with `dim_date` on `Date_Key` — producing one unified analytical dataframe (`df`). Before proceeding, the notebook validates the merged dataset by checking its shape, data types, missing values, and duplicate rows, ensuring that the feature engineering stage starts from a trustworthy foundation.

3. Data Preparation & Feature Engineering

Once merged and validated, the dataset is enriched with dozens of derived columns designed to capture time patterns, commercial efficiency, and momentum in the business. The date column is converted to a proper datetime type and the data is sorted chronologically before any rolling or lag calculation is performed, which is essential for time-aware features to be computed correctly per product.

Time-Based Features

- Year, Month, Month name, Quarter, ISO Week, Day, Day name
- Weekend flag (Saturday/Sunday) to capture weekly seasonality

Commercial Performance Ratios

- Conversion Rate (Units / Sessions)
- Click-Through Rate (Clicks / Impressions) and Cost-Per-Click (PPC Cost / Clicks)
- Revenue per Click and Revenue per Session
- Profit Margin (Net Profit / Revenue)
- Average Order Value, AOV (Revenue / Orders)
- Refund Rate (Refunded / Revenue)

Rolling & Momentum Features

- 7-day and 30-day rolling average Revenue per product
- 7-day rolling average Net Profit per product
- 7-day rolling average Orders per product

Lag & Growth Features

- 1-day and 7-day lagged Revenue, 1-day lagged Profit and Orders
- Revenue Growth, Profit Growth, and Orders Growth vs. previous period

Final Cleaning

- Infinite values replaced, remaining missing values filled with zero
- Cleaned, model-ready dataset exported to `ml_ready_data.csv`

4. Executive KPIs

Before the modeling stage, the notebook computes a set of headline business metrics that summarize overall performance at a glance: **Total Revenue**, **Total Net Profit**, **Total Orders**, **Average Net Margin**, and **Average Net ROI**. These KPIs give stakeholders an immediate, high-level read on business health before diving into the anomaly-level detail produced by the machine learning model. They are printed in a clean, formatted executive summary block so that anyone opening the notebook — technical or non-technical — can immediately grasp the scale of the business (total revenue and orders), its profitability (net margin and ROI), and, once the model has run, the number of anomalies detected during the reporting period.

5. Tools, Libraries & Data Quality Checks

Core Libraries Used

- `pandas` & `numpy` for data manipulation and numerical operations
- `matplotlib` & `plotly.express` / `graph_objects` for static and interactive visualizations
- `scikit-learn` for preprocessing (`StandardScaler`), dimensionality reduction (`PCA`), and modeling (`IsolationForest` , `Pipeline`)
- `joblib` for persisting the trained pipeline to disk
- `warnings` filter suppression to keep notebook output clean and readable for reviewers

Data Quality & Validation Steps

- Shape verification for every source table immediately after loading (row and column counts)
- Post-merge integrity check via `df.info()` to confirm data types and non-null counts
- Explicit checks for missing values and duplicate rows before any feature is engineered
- Conversion of the `Date` column to a proper datetime type, followed by chronological sorting
- Replacement of infinite values (produced by ratio-based features such as division by zero) with safe defaults
- Final null-handling pass (`fillna`) so the model-ready dataset contains no missing or infinite values

This combination of validation steps matters because the downstream model is sensitive to bad inputs: ratio features like Conversion Rate, CTR, or Refund Rate can easily produce infinite or undefined values whenever a denominator (sessions, clicks, or revenue) is zero, and an unscaled or corrupted feature can distort the distance-based logic that Isolation Forest relies on. Cleaning these edge cases before training is what keeps the anomaly scores meaningful and trustworthy.

5. The Model: Anomaly Detection with Isolation Forest

The core intelligence of this project is built on the **Isolation Forest** algorithm from scikit-learn — an unsupervised machine learning technique designed specifically for anomaly detection. Unlike supervised models, it does not require any pre-labeled "normal" versus "abnormal" examples; it learns the structure of the data itself and flags the points that do not fit the general pattern.

How the Algorithm Works, in Plain Terms

Isolation Forest builds a large number of random decision trees, each of which tries to isolate a single data point by repeatedly splitting the feature space at random thresholds. Data points that behave normally sit in dense, crowded regions of the feature space, so many splits are needed before they become isolated. Anomalous points, on the other hand, tend to be sparse outliers, so they can typically be isolated after only a handful of random splits. The fewer splits required to isolate a point, the higher its anomaly score — this simple geometric intuition is what makes the algorithm both fast and effective on high-dimensional business data.

Technical Pipeline (End-to-End)

- 1) Select 29 numeric features covering revenue, orders, clicks, advertising cost, efficiency ratios, and rolling/lag/growth signals
- 2) Standardize all features using `StandardScaler` so no single metric dominates due to scale
- 3) Train an `IsolationForest` model (300-500 estimators, contamination set to "auto" or 5%, fixed random seed for reproducibility)
- 4) Compute an Anomaly Score for every row using the model's negative sample score
- 5) Flag the top 2% highest-scoring rows as anomalies, and bucket the remaining scores into four risk levels via quantile-based segmentation

Risk Level	Interpretation
Low Risk	Sales behavior is fully consistent with normal historical patterns
Medium Risk	A mild deviation worth monitoring, but not urgent
High Risk	A clear, noticeable deviation from the expected pattern
Critical	A strong anomaly that warrants immediate investigation

The entire trained pipeline — the fitted `StandardScaler` together with the trained `IsolationForest` model — is serialized and saved to `anomaly_detection_pipeline.pkl` using `joblib`. This makes the solution production-ready: new incoming data can be scored instantly by loading this single file, with no need to retrain the model from scratch each time fresh data arrives.

6. Visual Outputs & Dashboards

The notebook generates a set of visualizations that translate the model's output into decision-ready insight:

- **Executive KPI summary:** total revenue, net profit, total orders, average margin, and average ROI presented together for a quick health check.
- **Revenue trend with anomalies overlaid:** a time-series chart of revenue with detected anomaly points highlighted directly on the timeline.
- **PCA visualization:** the 29-dimensional feature space is projected down to two principal components so that normal and anomalous points can be visually separated and inspected.
- **Anomaly distribution chart:** a breakdown of how many records fall into each risk category.
- **Top 10 anomalous products:** a ranked view of the products most frequently associated with flagged anomalies, useful for prioritizing investigation.
- **Monthly revenue trend and Revenue vs. Orders** comparison charts for broader business context.
- **Daily anomaly count chart:** the number of anomalies detected per day, helping identify whether issues are isolated events or part of a sustained pattern.

7. Model Evaluation & Practical Considerations

Because Isolation Forest is unsupervised, there is no ground-truth "anomaly" label to compute traditional accuracy, precision, or recall against. Instead, the notebook validates the model qualitatively and statistically: the distribution of anomaly scores is plotted to confirm a clear separation between the bulk of normal observations and the long tail of high-scoring outliers, and the PCA projection is used as a visual sanity check that flagged points genuinely sit apart from the main cluster of data rather than being artifacts of a single noisy feature. The `contamination` parameter (auto or 5%) directly controls how conservative or aggressive the flagging threshold is, and the top-2% cutoff on the anomaly score gives the business a tunable lever: it can be tightened to surface only the most extreme cases, or loosened to catch softer deviations earlier.

A few practical considerations are worth noting for anyone maintaining this pipeline going forward. First, because the model is trained on the full historical dataset at once, it should be periodically retrained (e.g., monthly or quarterly) as new data accumulates, so that its notion of "normal" evolves together with genuine business growth and seasonality rather than flagging legitimate growth as anomalous. Second, since features are scaled using a single fitted `StandardScaler`, that same scaler — saved inside the pipeline — must always be reused when scoring new data, rather than fitting a new scaler, to keep the feature space consistent between training and inference. Third, results should always be reviewed together with domain context: a spike flagged as "Critical" might reflect a data entry error, a flash sale, a marketplace outage, or a genuine fraud pattern, and the model's role is to prioritize attention, not to replace human judgment.

8. Recommended Next Steps

- Automate periodic retraining of the pipeline on a fixed schedule as new Gold Layer data lands, keeping the anomaly baseline current.
- Introduce a lightweight alerting layer (e.g., email or Slack notification) that fires automatically whenever a new record is classified as "Critical".
- Extend the feature set with marketplace-specific seasonality indicators (holidays, promotional calendars) to reduce false positives around known sales events.
- Layer a supervised model on top of the anomaly output once enough analyst-confirmed labels have been collected, enabling precision/recall tracking over time.
- Package the scoring step as a small reusable function or API endpoint around `anomaly_detection_pipeline.pkl` so it can be called directly from a BI dashboard or scheduled job.

9. Business Value & Conclusion

This solution converts raw, multi-marketplace transactional data into an intelligent early-warning system. Instead of manually reviewing thousands of rows across products, dates, and channels, the model automatically directs the team's attention to the roughly 2% of records that matter most, ranks them by severity, and visualizes them clearly. The result is a faster, more reliable decision-making process, reduced analyst workload, and earlier detection of revenue or profitability issues across a complex, multi-channel e-commerce operation. Combined with the recommended next steps above, this pipeline can evolve from a diagnostic notebook into a fully automated monitoring layer for the business.